

Programação Orientada a Objetos em Java: Polimorfismo em Métodos de Classes

Dani Juliano Czelusniak, Prof. Dr.

Atualizado em 2026

1 Introdução ao Polimorfismo

Na Programação Orientada a Objetos (POO), o termo **Polimorfismo** origina-se do grego *poli* (muitos) e *morphos* (formas). Em essência, refere-se à capacidade de um objeto assumir muitas formas, ou mais especificamente, à capacidade de um método possuir múltiplas implementações dependendo da classe do objeto que o invoca.

O polimorfismo é um dos pilares mais poderosos da POO, ao lado do Encapsulamento, da Herança e da Abstração. Ele permite que o desenvolvedor escreva um código mais genérico, flexível e fácil de manter. Em Java, o polimorfismo ocorre de forma muito natural através do uso de referências de superclasses (classes pai) apontando para instâncias de subclasses (classes filhas).

1.1 Tipos de Polimorfismo

Em Java, existem basicamente dois tipos de polimorfismo:

- **Polimorfismo Estático (Sobrecarga / *Overloading*):** Ocorre em tempo de compilação. É quando temos vários métodos com o mesmo nome na mesma classe, mas com assinaturas (parâmetros) diferentes.
- **Polimorfismo Dinâmico (Sobrescrita / *Overriding*):** Ocorre em tempo de execução. É quando uma subclasse provê uma implementação específica para um método que já foi declarado em sua superclasse. **Este é o foco principal do nosso material de estudo.**

1.2 O Mecanismo de Despacho Dinâmico

Quando utilizamos uma variável do tipo de uma superclasse para instanciar um objeto de uma subclasse (ex: `Animal a = new Cachorro();`), o compilador Java confia no tipo da variável de referência (`Animal`) para verificar se o método existe. No entanto, durante a **execução** do programa, a Máquina Virtual Java (JVM) verifica o tipo real do objeto instanciado na memória (`Cachorro`) e aciona o método correspondente a essa classe específica. Esse processo é chamado de *Dynamic Method Dispatch* (Despacho Dinâmico de Métodos).

2 Exemplo 1: Método Vazio na Superclasse

Em muitos projetos, a superclasse representa um conceito genérico que deve ser capaz de executar uma ação, mas a maneira exata como essa ação será executada varia para cada subclasse. Uma abordagem comum é declarar o método na superclasse de forma vazia (ou como abstrato, embora aqui demonstraremos com o corpo vazio) e sobrescrevê-lo nas subclasses.

2.1 Diagrama de Classes

Abaixo, ilustramos de forma didática¹ um o relacionamento de herança e polimorfismo entre um sistema de `Notificacao`. A classe pai possui um método `enviar()` vazio.

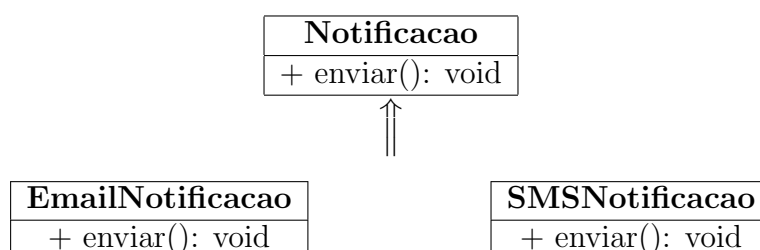


Figura 1: Fragmento de diagrama de classes com herança e reescrita do método `enviar()`.

2.2 Implementação em Java

Note que na classe `Notificacao`, o método `enviar()` possui chaves `{}`, mas nenhum código interno. Ele existe apenas para garantir o contrato, permitindo o polimorfismo.

```
1 class Notificacao {
2     // Metodo declarado na classe pai vazio. Serve apenas de "
3     molde".
4     // (Nota: Em cenarios reais, frequentemente marcamos como '
5     abstract')
6     public void enviar() {
7         // Corpo vazio
8     }
9 }
10
11 class EmailNotificacao extends Notificacao {
12     // Sobrescrevendo o metodo vazio com comportamento especifico
13     @Override
14     public void enviar() {
15         System.out.println("Comportamento A: Enviando um E-mail
16         com anexo...");
17     }
18 }
19
20 class SMSNotificacao extends Notificacao {
21     // Sobrescrevendo o metodo vazio com comportamento especifico
```

¹Somente no formato de um “recorde de classes” de um diagrama maior. Essa figura não está seguindo o rigor da notação UML

```

19     @Override
20     public void enviar() {
21         System.out.println("Comportamento B: Enviando um SMS de
22         texto curto...");
23     }
24 }
25 public class Main {
26     public static void main(String[] args) {
27         System.out.println("--- Teste de Polimorfismo (Metodo
28         Vazio) ---");
29
30         // Uso de variaveis da Superclasse apontando para
31         Subclasses
32         Notificacao n1 = new EmailNotificacao();
33         Notificacao n2 = new SMSNotificacao();
34
35         // A magia do Polimorfismo: a mesma chamada de metodo
36         // gera comportamentos completamente diferentes!
37         n1.enviar();
38         n2.enviar();
39     }
40 }

```

Listing 1: Polimorfismo com método pai vazio

3 Exemplo 2: Método com Implementação Sobrescrito na Subclasse

Outro cenário vital é quando a superclasse já possui um comportamento padrão (implementação de código) para um método, mas uma subclasse específica precisa alterar ou expandir esse comportamento. Neste caso, utilizamos a anotação `@Override` para substituir a lógica padrão, exercendo o polimorfismo.

3.1 Diagrama de Classes

O sistema² abaixo modela Contas Bancárias. O método `aplicarRendimento()` possui uma regra padrão na classe pai, mas é substituído na classe filha para garantir taxas diferentes.

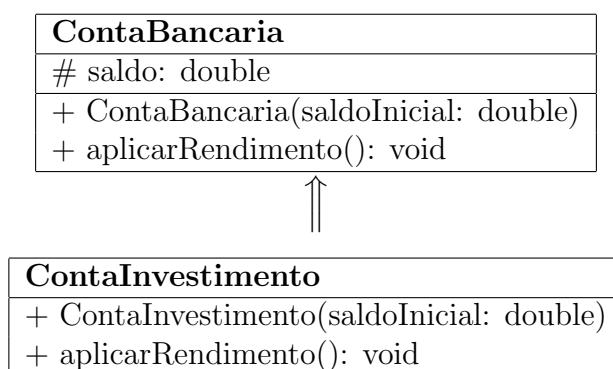


Figura 2: Diagrama: Método `aplicarRendimento()` reescrito na subclasse.

3.2 Implementação em Java

No código a seguir, `ContaBancaria` define um rendimento de 1%. Já a `ContaInvestimento` não aceita esse padrão e implementa (sobrescreve) o método para aplicar 5%.

```
1 class ContaBancaria {
2     protected double saldo;
3
4     public ContaBancaria(double saldoInicial) {
5         this.saldo = saldoInicial;
6     }
7
8     // Metodo com implementacao em codigo fonte (Regra padrao)
9     public void aplicarRendimento() {
10         this.saldo += this.saldo * 0.01; // Rendimento de 1%
11         System.out.println("Rendimento padrao aplicado. Novo
12         saldo: R$ " + this.saldo);
13     }
14 }
15 class ContaInvestimento extends ContaBancaria {
```

²Assim como na figura anterior, esse é somente um “recorde de classes” de um diagrama maior. Essa figura não está seguindo o rigor da notação UML

```

16
17     public ContaInvestimento(double saldoInicial) {
18         super(saldoInicial); // Chama o construtor da classe pai
19     }
20
21     // Sobrescrita do metodo existente. Ignoramos a regra padrao
22     // do pai
23     // e criamos uma regra totalmente nova para o investimento.
24     @Override
25     public void aplicarRendimento() {
26         this.saldo += this.saldo * 0.05; // Rendimento de 5%
27         System.out.println("Rendimento de investimento aplicado!
28         Novo saldo: R$ " + this.saldo);
29     }
30 }
31
32 public class Main {
33     public static void main(String[] args) {
34         System.out.println("--- Teste de Polimorfismo (Metodo
35         Implementado) ---");
36
37         ContaBancaria contaComum = new ContaBancaria(1000.0);
38         ContaBancaria contaEspecial = new ContaInvestimento
39         (1000.0);
40
41         // Aciona o metodo original da classe pai
42         contaComum.aplicarRendimento();
43
44         // Polimorfismo ativado: Aciona o metodo reescrito na
45         // classe filha
46         contaEspecial.aplicarRendimento();
47     }
48 }

```

Listing 2: Polimorfismo substituindo comportamento padrão

4 Lista de Exercícios Práticos

A fim de consolidar os conceitos de Polimorfismo Dinâmico abordados, resolva e analise os 6 exercícios a seguir, que estão classificados por nível de dificuldade.

4.1 Nível Fácil

Exercício 1: Instrumentos Musicais

Enunciado: Crie uma classe base `Instrumento` com o método vazio `tocar()`. Crie duas subclasses, `Violao` e `Piano`, que herdam de `Instrumento` e sobrescrevem o método `tocar()` imprimindo mensagens como "Som de cordas do violão" e "Som das teclas do piano". No método principal, instancie ambos sob variáveis do tipo `Instrumento` e chame o método polimórfico.

```
1 class Instrumento {
2     public void tocar() {
3         // Metodo vazio na superclasse
4     }
5 }
6
7 class Violao extends Instrumento {
8     @Override
9     public void tocar() {
10         System.out.println("Tocando Violao: plin plin plin...");
11     }
12 }
13
14 class Piano extends Instrumento {
15     @Override
16     public void tocar() {
17         System.out.println("Tocando Piano: dom dom dom...");
18     }
19 }
20
21 public class Main {
22     public static void main(String[] args) {
23         Instrumento i1 = new Violao();
24         Instrumento i2 = new Piano();
25
26         // Chamada polimorfica
27         i1.tocar();
28         i2.tocar();
29     }
30 }
```

Listing 3: Resolução: Exercício 1 - Fácil

Exercício 2: Saudações Internacionais

Enunciado: Crie uma classe `Pessoa` com um método `saudar()` que imprime "Olá". Crie subclasses `Brasileiro` e `Ingles`. O `Brasileiro` deve manter o comportamento da

superclasse (não precisa sobrescrever), enquanto o `Ingles` deve sobrescrever e imprimir "Hello".

```
1 class Pessoa {
2     // Metodo com implementacao padrao
3     public void saudar() {
4         System.out.println("Ola!");
5     }
6 }
7
8 class Brasileiro extends Pessoa {
9     // Nao sobrescreve, herda o comportamento natural
10 }
11
12 class Ingles extends Pessoa {
13     @Override
14     public void saudar() {
15         System.out.println("Hello!");
16     }
17 }
18
19 public class Main {
20     public static void main(String[] args) {
21         Pessoa p1 = new Brasileiro();
22         Pessoa p2 = new Ingles();
23
24         p1.saudar(); // Usa implementacao do pai
25         p2.saudar(); // Usa implementacao sobrescrita (
26         Polimorfismo)
27     }
28 }
```

Listing 4: Resolução: Exercício 2 - Fácil

4.2 Nível Médio

Exercício 3: Loja de Roupas (Cálculo de Desconto)

Enunciado: Implemente a classe `Produto` contendo o atributo `preco` e o método `calcularPrecoFinal()` que simplesmente retorna o valor original. Crie a subclasse `ProdutoEmPromocao` que sobrescreve este método para retornar o preço com 20% de desconto. Teste-os no método `main`.

```
1 class Produto {
2     protected double preco;
3
4     public Produto(double preco) {
5         this.preco = preco;
6     }
7
8     // Comportamento padrao: sem desconto
9     public double calcularPrecoFinal() {
10         return this.preco;
11     }
12 }
13
14 class ProdutoEmPromocao extends Produto {
15     public ProdutoEmPromocao(double preco) {
16         super(preco);
17     }
18
19     @Override
20     public double calcularPrecoFinal() {
21         // Aplica 20% de desconto polimorficamente
22         return this.preco * 0.80;
23     }
24 }
25
26 public class Main {
27     public static void main(String[] args) {
28         Produto calca = new Produto(100.0);
29         Produto camisaPromo = new ProdutoEmPromocao(100.0);
30
31         System.out.println("Calca (Normal): R$ " + calca.
32             calcularPrecoFinal());
33         System.out.println("Camisa (Promo): R$ " + camisaPromo.
34             calcularPrecoFinal());
35     }
36 }
```

Listing 5: Resolução: Exercício 3 - Médio

Exercício 4: Frotas de Veículos e Reuso de Código

Enunciado: Crie a classe `Veiculo` com o método `acelerar()` que imprime "O veiculo está acelerando". Crie a subclasse `MotoEsportiva` que sobrescreve o método. O detalhe importante: a `MotoEsportiva` deve imprimir "Empinando a roda..." E TAMBÉM executar o comportamento padrão de aceleração do pai usando a palavra-chave `super`.

```

1 class Veiculo {
2     public void acelerar() {
3         System.out.println("O veiculo esta acelerando normalmente
4     };
5 }
6
7 class MotoEsportiva extends Veiculo {
8     @Override
9     public void acelerar() {
10        System.out.println("Empinando a roda dianteira...");
11        // Reutilizando o codigo da superclasse
12        super.acelerar();
13    }
14 }
15
16 public class Main {
17     public static void main(String[] args) {
18         Veiculo v1 = new MotoEsportiva();
19
20         System.out.println("Acao da Moto:");
21         v1.acelerar();
22     }
23 }

```

Listing 6: Resolução: Exercício 4 - Médio

4.3 Nível Difícil

Exercício 5: Sistema de Pagamento Genérico (Arrays e Laços)

Enunciado: Polimorfismo é extremamente útil em laços de repetição. Crie uma classe `MetodoPagamento` com o método vazio `processarPagamento(double valor)`. Crie três filhas: `Pix`, `CartaoCredito` e `Boleto`, cada uma com suas taxas (+0%, +5%, e taxa fixa de R\$2, respectivamente). No main, crie um vetor do tipo `MetodoPagamento[]` contendo as três formas, itere sobre o vetor e processe um pagamento de R\$ 100 em cada um deles sem utilizar estruturas condicionais (if ou switch).

```
1 class MetodoPagamento {
2     public void processarPagamento(double valor) {} // Metodo
    vazio
3 }
4
5 class Pix extends MetodoPagamento {
6     @Override
7     public void processarPagamento(double valor) {
8         System.out.println("Pagamento PIX processado: R$ " +
    valor + " (Sem taxas)");
9     }
10 }
11
12 class CartaoCredito extends MetodoPagamento {
13     @Override
14     public void processarPagamento(double valor) {
15         double comTaxa = valor + (valor * 0.05);
16         System.out.println("Pagamento Cartao processado: R$ " +
    comTaxa + " (+5% taxa)");
17     }
18 }
19
20 class Boleto extends MetodoPagamento {
21     @Override
22     public void processarPagamento(double valor) {
23         double comTaxa = valor + 2.00;
24         System.out.println("Pagamento Boleto processado: R$ " +
    comTaxa + " (+R$2 taxa fixa)");
25     }
26 }
27
28 public class Main {
29     public static void main(String[] args) {
30         // Criacao de uma estrutura de dados padronizada pela
    classe pai
31         MetodoPagamento[] pagamentos = new MetodoPagamento[3];
32         pagamentos[0] = new Pix();
33         pagamentos[1] = new CartaoCredito();
34         pagamentos[2] = new Boleto();
35
36         // Iteracao generica! O polimorfismo decide a regra
    matematica em tempo de execucao.
```

```
37      // Nao e necessario saber o tipo especifico no laço.  
38      System.out.println("--- Processamento em Lote ---");  
39      for (int i = 0; i < pagamentos.length; i++) {  
40          pagamentos[i].processarPagamento(100.0);  
41      }  
42  }  
43 }
```

Listing 7: Resolução: Exercício 5 - Difícil

Exercício 6: Jogo de RPG (Ataques Especiais Baseados no Estado)

Enunciado: Crie a classe `Personagem` com vida e força. Crie o método com implementação `atacar(Personagem alvo)`, que reduz a vida do alvo com base na força. Crie as subclasses `Guerreiro` e `Assassino`. O `Guerreiro` ataca duas vezes no mesmo turno (usando `super` duas vezes). O `Assassino` possui uma regra crítica: se a força for maior que 10, causa triplo de dano (sobrescrevendo a lógica sem `super`); caso contrário, causa o dano normal. Simule um pequeno combate no `main`.

```
1 class Personagem {
2     protected String nome;
3     protected int vida;
4     protected int forca;
5
6     public Personagem(String nome, int vida, int forca) {
7         this.nome = nome; this.vida = vida; this.forca = forca;
8     }
9
10    public void sofrerDano(int dano) {
11        this.vida -= dano;
12        System.out.println(this.nome + " sofreu " + dano + " de
13        dano. Vida restante: " + this.vida);
14    }
15
16    public void atacar(Personagem alvo) {
17        System.out.println(this.nome + " ataca " + alvo.nome + "
18        causando " + this.forca + " de dano!");
19        alvo.sofrerDano(this.forca);
20    }
21 }
22
23 class Guerreiro extends Personagem {
24     public Guerreiro(String nome, int vida, int forca) { super(
25     nome, vida, forca); }
26
27     @Override
28     public void atacar(Personagem alvo) {
29         System.out.println(this.nome + " usa ATAQUE DUPLO!");
30         super.atacar(alvo); // Ataque 1
31         super.atacar(alvo); // Ataque 2
32     }
33 }
34
35 class Assassino extends Personagem {
36     public Assassino(String nome, int vida, int forca) { super(
37     nome, vida, forca); }
38
39     @Override
40     public void atacar(Personagem alvo) {
41         if (this.forca > 10) {
42             System.out.println(this.nome + " executa ASSASSINATO
43             CRITICO (Triplo dano)!");
44             alvo.sofrerDano(this.forca * 3); // Regra reescrita
```

```

40     totalmente
41     } else {
42         System.out.println(this.nome + " ataca fracamente..."
43     );
44         super.atacar(alvo); // Recai para a regra original
45     }
46 }
47
48 public class Main {
49     public static void main(String[] args) {
50         Personagem boss = new Personagem("Chefao Orc", 100, 5);
51         Personagem heroi1 = new Guerreiro("Arthur", 50, 10);
52         Personagem heroi2 = new Assassino("Ezio", 40, 15);
53
54         // Despacho dinamico selecionara a animacao/dano correto
55         de cada classe filha
56         heroi1.atacar(boss);
57         System.out.println("-----");
58         heroi2.atacar(boss);
59     }
60 }

```

Listing 8: Resolução: Exercício 6 - Difícil